Las seis pruebas del profesor, en NOVA

Esta carpeta es **aditiva**: no modifica ningún archivo del proyecto entregado. Se apoya en `test/run_rtl_test.sh`, que ya venía, y escribe solo en su propio directorio temporal, que borra al salir.

Al lado de cada `.nv` está el `.py` original del profesor, para poder compararlos en pantalla.

Cómo correrlas

Desde la raíz del proyecto:

```
make                                  # construye el compilador
sh pruebas_profesor/correr.sh         # las seis sobre el RTL de CERBERO
```

Termina con `6 de 6 coinciden con el original en Python; 0 fallan`.

Una sola, para mostrarla con calma:

```
sh test/run_rtl_test.sh ./novac pruebas_profesor/Test3.nv 110 ARQUITECTURA/src/cpu
```

Y para ver las representaciones intermedias de cualquiera de ellas:

```
./novac --tokens pruebas_profesor/Test6.nv     # tokens, con línea y columna
./novac -t       pruebas_profesor/Test6.nv     # árbol sintáctico con color
./novac -m       pruebas_profesor/Test6.nv     # tabla de símbolos con direcciones
./novac --ir     pruebas_profesor/Test6.nv     # código de tres direcciones
./novac -s       pruebas_profesor/Test6.nv -o /tmp/t6.nvx   # ensamblador
./novac -p       pruebas_profesor/Test6.nv     # bundles con dirección y bytes
```

Qué resultado da cada una

Archivo	Python original	NOVA sobre CERBERO
`Test1.nv`	`28`	retorna 28
`Test2.nv`	biblioteca, sin salida	se importa desde `Test3.nv`
`Test2_demo.nv`	`65`	retorna 65
`Test3.nv`	`(100, 10)`	imprime 100 y 10, retorna 110
`Test4.nv`	`[[1,4],[2,5],[3,6]]`	imprime los 6, retorna 21
`Test5.nv`	`[[19,22],[43,50]]`	imprime los 4, retorna 134
`Test6.nv`	`55`	retorna 55

CERBERO no tiene puertos de entrada y salida, así que `scribe` escribe en la zona de salida de DMEM y el testbench la lee desde la memoria del procesador. El valor de retorno queda en `R6`. Por eso los programas que en Python imprimen una lista aquí imprimen sus elementos y además retornan algo que resuma el resultado, para poder verificarlo desde `R6`.

Lo que hubo que resolver al traducir

`global suma` (Test1 y Test3). NOVA no admite variables globales mutables: el error `E320` exige que toda global sea `constans`. Es una decisión del diseño del lenguaje, no una carencia — CERBERO no tiene gestión dinámica de memoria, y el lenguaje mantiene todo el estado mutable en el marco de pila, que es lo que permite calcular los desplazamientos en tiempo de compilación. El acumulador se vuelve local y se devuelve.

`divisor=2`**, valor por omisión** (Test1). NOVA no tiene parámetros con valor por defecto. Se pasa el `2` explícitamente en la llamada.

Devolver la tupla `(suma, multiplicacion)` (Test3). NOVA devuelve un solo valor. El par sale por un arreglo de salida, que es el mismo patrón que ya usa `test/ejemplos/busqueda_matriz.nv` para devolver la posición encontrada.

`transpuesta = []` **y** `.append()` (Test4 y Test5). NOVA tiene arreglos de tamaño fijo, porque el ISA no tiene asignación dinámica. La matriz de salida se recibe ya dimensionada como parámetro.

**`valorMaximo/2` ** (Test3). En Python 3 es división real y da `5.0`; en NOVA es división entera y da `5`. Para los datos de la prueba el resultado coincide. NOVA solo tiene el tipo `integer`, declarado así en la especificación.

Un detalle para no tropezar en la revisión

`Test2.nv` es un **módulo de biblioteca**: no tiene `principale`, igual que el `Test2.py` original, cuyo `print` está comentado. Compilarlo suelto da un error de enlace, que es el comportamiento correcto:

```
$ ./novac pruebas_profesor/Test2.nv
error[E405]: no hay 'principale': un modulo de biblioteca no puede ser el
punto de entrada de un ejecutable
```

Para demostrarlo funcionando está `Test2_demo.nv`, que lo importa y lo llama. Y `Test3.nv` es el que conviene mostrar: importa `Test2`, lo enlaza y corre. Los módulos múltiples son justamente lo que la sección 9.1 del enunciado dice que **no** hacía falta implementar.

Guion para la revisión: las fases sobre la prueba del profesor

Lo más convincente es tomar una de sus propias pruebas y recorrer la cadena completa. Con `Test6.nv` (fibonacci) cabe en una pantalla.

Fases 1 a 4

```
./novac --tokens pruebas_profesor/Test6.nv   # 64 tokens, con línea y columna
./novac -t       pruebas_profesor/Test6.nv   # árbol con conectores y color
```

```
./novac -m      pruebas_profesor/Test6.nv   # símbolos con dirección IMEM y R14
./novac --ir    pruebas_profesor/Test6.nv   # código de tres direcciones
```

En la tabla de símbolos se ve `fibonacci IMEM @0x0070` y `n R14 @-4 -> +0(r14)`: la dirección de código y el desplazamiento en el marco, que es lo que pide la sección 2.3 del enunciado.

Fase 5: cálculo de saltos, verificable a mano

```
./novac -s pruebas_profesor/Test6.nv -o /tmp/t6.nvx > /dev/null
grep -n "br\." /tmp/t6.asm
grep "IMEM bundle" /tmp/t6.asm
```

El `si (n <= 1)` de la línea 4 produce tres saltos. Tomando el último:

```
    br.eq L0    ;  // 0x0250        <- el salto está en el bundle 37
//   L0           0x02E0  IMEM bundle 46   <- la etiqueta quedó en el 46
```

El desplazamiento del formato tipo C es `destino − origen − 1`, donde el −1 es el delay slot:

```
46 − 37 − 1 = 8
```

Fase 6: y ese 8 está en los bits

```
./novac -p pruebas_profesor/Test6.nv | grep "br.eq"
```

```
0x0250  4  5  ... | br.eq L0 |  08240000000000000000000000000000
```

El slot S4 son los 16 bits bajos del bundle, que el listado imprime primero en little endian: `0824` leído como palabra es `0x2408`. Sus diez bits bajos son `0x008`, **el 8 calculado arriba**. Los tres bits de condición son `001` = EQ.

Los tres saltos del programa, comprobados igual:

Salto	En	Destino	`d − o − 1`	S4	Desplazamiento en los bits
`br.ge __c0_si`	0x0170	0x01E0	6	`0x3006`	6
`br.al __c0_fin`	0x01C0	0x0210	4	`0x2004`	4
`br.eq L0`	0x0250	0x02E0	8	`0x2408`	8

Y corre en el procesador

```
sh test/run_rtl_test.sh ./novac pruebas_profesor/Test6.nv 55 ARQUITECTURA/src/cpu
```

Si pregunta por las filas con `-` en el listado

En la columna de línea y columna, algunas filas muestran `-`. Son los bundles que no vienen de ninguna línea del fuente: el arranque, el prólogo y el epílogo de cada función, el paso de argumentos y las rutinas de soporte como `__scribe_int`. Se deja `-` en vez de inventarles una posición.

El lado de la arquitectura

Las mismas seis pruebas recorriendo la cadena completa hasta el testbench de CERBERO, y con la comprobación cruzada contra el ensamblador independiente de CE 4301:

```
sh pruebas_profesor/arquitectura.sh
```

Termina con `6 de 6 identicos bit a bit con cerbero_asm.py; 0 distintos`.

No escribe nada dentro de `ARQUITECTURA/`. Los artefactos quedan en `pruebas_profesor/.salida_arq/` y a `make correr` se le pasa la ruta del `.s`, porque la variable `PROG` acepta una ruta cuando lleva `/`.

Una prueba paso a paso

Cada comando va en **una sola línea**, a propósito. Si se parten con `\` al final y al pegarlos queda un espacio después de la barra, la shell se come el argumento siguiente y el comando falla con `the following arguments are required: --imem`.

```
T=Test3                                  # la que importa otro módulo
O=pruebas_profesor/.salida_arq
RAIZ=$(pwd)
rm -rf $O && mkdir -p $O                 # sin esto, el paso 4 puede comparar
archivos viejos

# 1. NOVA produce el ensamblador y el binario de una sola pasada
./novac pruebas_profesor/$T.nv -s -o $O/$T.nvx > /dev/null

# 2. La imagen IMEM que arma NOVA desde su propio binario
python3 ARQUITECTURA/tools/novac_image.py $O/$T.nvx --imem $O/$T.nova.hex --dmem
$O/$T.nova_d.hex

# 3. La imagen que arma el ensamblador de CE 4301 desde el texto
sed 's/\$/_/g' $O/$T.asm > $O/$T.s
python3 ARQUITECTURA/tools/cerbero_asm.py $O/$T.s -o $O/$T.cerbero.hex --datos
$O/$T.cerbero_d.hex --bundles 2048 --palabras 16384
```

```
# 4. Las dos codificaciones, bundle por bundle
diff -i $O/$T.nova.hex $O/$T.cerbero.hex && echo "IDENTICOS en los 2048 bundles"

# 5. Y sobre el testbench de CERBERO
( cd ARQUITECTURA/src/build && make correr PROG=$RAIZ/$O/$T.s )
```

El `rm -rf $O` del principio importa más de lo que parece: el paso 4 compara dos archivos, y si los pasos 2 o 3 fallan sin que uno lo note, el `diff` puede comparar los que dejó una corrida anterior y responder `IDENTICOS` sin haber generado nada. Borrar la carpeta antes hace que un fallo se note.

El pipeline, ciclo por ciclo

Este bloque se basta solo: genera el `.s` y después lo traza. Hace falta porque el paso a paso de arriba empieza con `rm -rf $O` y solo deja el `.s` de la prueba que se estaba mirando; si se salta la generación, `make correr` responde `no se encontro ... ni aqui ni en tools/programas`.

```
T=Test6
O=pruebas_profesor/.salida_arq
RAIZ=$(pwd)
mkdir -p $O
./novac pruebas_profesor/$T.nv -s -o $O/$T.nvx > /dev/null
sed 's/\$/_/g' $O/$T.asm > $O/$T.s
( cd ARQUITECTURA/src/build && make correr PROG=$RAIZ/$O/$T.s TRAZA=1-12 )
```

Dibuja cada bundle pasando por IF, ID, EX, ME y WB, marca el salto y su delay slot, y lista los eventos de escritura a registro con su número de ciclo. Es la forma más directa de mostrar la latencia de tres bundles que el compilador respeta.

Por qué el `sed` del paso 3

NOVA decora los símbolos de módulo con `$`, como `m_Test2_934d2dec$maximo_lista`. La gramática de etiquetas de `cerbero_asm.py` no acepta ese carácter, así que rechaza el archivo:

```
error: linea 675: el bundle debe terminar en ';'
      m_Test2_934d2dec$maximo_lista:
```

Cambiar `$` por `_` es un renombre uniforme: no toca ni un opcode, ni un registro, ni una dirección. Con eso las seis imágenes salen idénticas, incluidas las dos que importan otro módulo. Solo hace falta para `Test2_demo` y `Test3`; en las otras cuatro el `sed` no cambia nada.

Es una incompatibilidad real entre los dos proyectos, de un solo carácter, y vale la pena mencionarla: lo correcto sería acordar con CE 4301 si NOVA cambia el separador o si su ensamblador admite `$` en las etiquetas.